

BINX TECH • BACKEND DEVELOPMENT INTERNSHIP PROGRAM (.NET)

## WEEK 6

### SPRINT 1 — CORE ROUTES & DATABASE SCHEMA

*The first Phase 3 sprint: planning the capstone project properly, then building its core routes and database schema from scratch.*

Week 6 opens Phase 3's four-sprint structure. Interns run a full Sprint Planning session, design the capstone project's complete database schema, implement it in Entity Framework Core with the Fluent API and code-first migrations, and build the first set of core catalog and order-creation routes — closing with a real GitHub pull request, a mentor code review, and a Postman-driven Sprint Review.

PHASE 3 — SPRINT 1 • 40 HOURS • 5 TRAINING DAYS • CORE ROUTES &amp; SCHEMA

Prepared by BinX Tech

Palestine | Nablus

A large, light blue watermark logo for BinX Tech, featuring the word 'BinX' in a large, bold, sans-serif font, with 'TECH' in a smaller, blue, sans-serif font below it.



## 1. Week 6 Overview

| Field              | Detail                                                                                          |
|--------------------|-------------------------------------------------------------------------------------------------|
| Week               | Week 6 of 10 — Phase 3: Applied Project Work, Sprint 1                                          |
| Total Hours        | 40 hours (full-time track) / 20 hours (part-time track, Weeks 11-12 combined)                   |
| Format             | On-site / Remote / Hybrid — code committed to GitHub, reviewed via pull request                 |
| Focus              | Project planning & Sprint 1: core routes and database schema                                    |
| Prerequisite       | Week 5 completed — chosen project, testing foundation, centralized error handling               |
| Mentor Supervision | Mentor Code Review mid-sprint via GitHub pull request; Sprint Review and Retrospective on Day 5 |

## 2. Sprint 1 Structure

Week 6 is Sprint 1 of the Phase 3 capstone project, following the program's standard sprint cycle for every remaining week of the project:

| Event                           | Description                                                                                                             |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Sprint Planning (Day 1)         | Intern selects tasks from the backlog, estimates effort, and commits to a sprint goal including specific API endpoints. |
| Daily Stand-up (Daily)          | 3-minute update: what was completed, what is next, and any blockers.                                                    |
| Mentor Code Review (Mid-sprint) | Mentor reviews committed code via GitHub pull request; leaves structured inline comments on API design and logic.       |
| Sprint Review (Day 5)           | Intern demos completed API features via Postman to mentor. Unfinished tasks move to the next sprint backlog.            |
| Sprint Retrospective (Day 5)    | What went well, what to improve, and one specific action item for the next sprint.                                      |

## 3. Week 6 Learning Objectives

- Run a full Sprint Planning session and build a realistic, sized backlog.
- Design and diagram the capstone project's complete database schema.
- Implement the full EF Core data model using the Fluent API and code-first migrations.
- Build paginated, filterable catalog endpoints projecting to DTOs.
- Implement write operations with real business logic, wrapped in database transactions where needed.
- Open a clean pull request and participate in a structured mentor code review.



## 4. Daily Schedule

| Day   | Hours | Topic Focus                                                                          |
|-------|-------|--------------------------------------------------------------------------------------|
| Day 1 | 8 hrs | Sprint 1 planning and project database design                                        |
| Day 2 | 8 hrs | Building the EF Core data model and migrations                                       |
| Day 3 | 8 hrs | Implementing core routes I: catalog and read operations                              |
| Day 4 | 8 hrs | Implementing core routes II: write operations and business logic; Mentor Code Review |
| Day 5 | 8 hrs | Sprint Review, Postman demo, and Retrospective                                       |

## 5. Day-by-Day Curriculum

### Day 1 — Sprint 1 Planning & Project Database Design — 8 hours

#### Learning Objectives

- Run a Sprint Planning session and translate it into a sized, realistic backlog.
- Design the capstone project's complete database schema.
- Finalize the schema as a documented ERD.

#### Key Topics

- Running Sprint Planning
- Designing the capstone's full schema
- Finalizing the ERD
- Sizing tasks realistically

#### Lesson Content

##### 1.1 Running Sprint Planning

Sprint Planning opens every Phase 3 sprint with a single question: what does “done” look like for this week? For Sprint 1, done means a complete database schema, applied migrations, and a working set of core routes covering the project's primary catalog and order-creation flow. The sprint goal gets written as one sentence at the top of the backlog, and every task added afterward should visibly serve it.

##### 1.2 Designing the Capstone's Full Schema

Unlike Week 3's small practice schema, Sprint 1 designs the entire capstone project's data model in one pass — every entity the professional baseline will eventually need, even if some won't be touched by an API endpoint until a later sprint. For an e-commerce backend, that means Products, Categories, Customers, Orders, OrderItems, and a Cart or CartItems table, with every relationship between them mapped out before any C# code is written.

##### 1.3 Finalizing the ERD

The ERD produced this week is the same one the Week 9 Definition of Done audit will check for — worth building carefully and keeping updated as the schema evolves across sprints, rather than treating it as a one-time Day 1 exercise. A schema diagram that falls out of sync with the actual database quickly becomes actively misleading rather than merely outdated.



```
Categories(Id PK, Name)
Products(Id PK, CategoryId FK, Name, Price, StockQuantity)
Customers(Id PK, UserId FK -> AspNetUsers.Id)
Orders(Id PK, CustomerId FK, OrderDate, Status, Total)
OrderItems(Id PK, OrderId FK, ProductId FK, Quantity, UnitPrice)
```

#### **1.4 Sizing Tasks Realistically**

A backlog task like “build the catalog” is too large to size or track meaningfully — breaking it into “implement Product entity and migration,” “implement paginated GET /products,” “implement GET /products/{id}” gives each task a clear, checkable definition of done. Sizing tasks to roughly half a day to a day each keeps the daily stand-up meaningful — genuine progress or a genuine blocker to report, rather than “still working on the same big thing.”

*A sprint with no written backlog tends to expand to fill the week with whatever feels most interesting moment to moment. Fifteen minutes of planning on Day 1, with tasks sized to about a day each, saves far more time than it costs across the week.*

#### **Hands-On Lab: Sprint 1 Planning & Full Schema Design**

1. Write a one-sentence Sprint 1 goal and set up a backlog board in Notion or Trello.
2. List every entity your chosen capstone project will need across its full professional baseline, not just this sprint's scope.
3. Design the complete schema, applying normalization principles from Week 3.
4. Diagram the finalized schema as an ERD using a diagramming tool or database GUI client.
5. Break Sprint 1's specific scope into sized backlog tasks of roughly half a day to a day each.

#### **Tools Used**

Notion or Trello • SQL Server Management Studio, Azure Data Studio, or dbdiagram.io

### **Day 2 — Building the EF Core Data Model & Migrations — 8 hours**

#### **Learning Objectives**

- Model the full capstone domain as EF Core entity classes.
- Configure relationships explicitly using the Fluent API.
- Seed initial reference data and apply the migration.

#### **Key Topics**

- Modeling the full domain in code
- Configuring relationships with the Fluent API
- Seeding initial data
- Applying and reviewing the migration

#### **Lesson Content**

##### **2.1 Modeling the Full Domain in Code**

Every entity from Day 1's ERD becomes a C# class with properties matching its columns and navigation properties representing its relationships — an Order has a Customer navigation property and a collection of OrderItems, mirroring the foreign keys drawn in the diagram. Keeping entity classes focused on data shape, with business logic living in service classes rather than the entities themselves, keeps the model layer simple and predictable as the project grows.



## 2.2 Configuring Relationships with the Fluent API

While EF Core can infer many relationships by convention, the Fluent API (configured in `OnModelCreating`) makes relationships, required fields, and constraints explicit rather than implicit — useful whenever a relationship's cardinality or delete behavior isn't the default EF Core would guess. Explicitly configuring cascade delete behavior, in particular, avoids the common surprise of an unexpected mass deletion when a parent record is removed.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>()
        .HasMany(o => o.Items)
        .WithOne(i => i.Order)
        .HasForeignKey(i => i.OrderId)
        .OnDelete(DeleteBehavior.Cascade);

    modelBuilder.Entity<Product>()
        .Property(p => p.Price)
        .HasColumnType("decimal(18,2)");
}
```

## 2.3 Seeding Initial Data

Seed data — a few starter categories, a couple of sample products — gives the API something real to return immediately after the first migration, without needing to manually insert test data through Postman before any endpoint can be demoed. EF Core's `HasData` method in `OnModelCreating` (or a small seeding method run at startup) is the standard place for this kind of fixed, known-in-advance data.

## 2.4 Applying and Reviewing the Migration

“dotnet ef migrations add InitialSchema” generates the migration from the full entity set, and it's worth actually reading the generated migration file before applying it — confirming the expected tables, columns, and foreign keys are all present, and that nothing unexpected (an accidental cascade delete, a missing index) slipped in silently. “dotnet ef database update” then applies it.

*Reviewing a generated migration file before applying it is a five-minute habit that catches schema mistakes before they reach a real database. Skipping this review is how an unintended cascade delete or a missing NOT NULL constraint quietly ships.*

### Hands-On Lab: Build the Full EF Core Model

1. Implement entity classes for every table in your Day 1 ERD, including navigation properties.
2. Configure at least 2 relationships explicitly via the Fluent API, including one delete behavior decision.
3. Add seed data for at least one reference table (e.g. categories) using `HasData`.
4. Generate the initial migration and review the generated file before applying it.
5. Apply the migration and confirm the schema in a database GUI client matches your ERD.

### Tools Used

Entity Framework Core • SQL Server



## Day 3 — Implementing Core Routes I: Catalog & Read Operations — 8 hours

### Learning Objectives

- Implement paginated list endpoints rather than returning entire tables at once.
- Support filtering and sorting through query parameters.
- Project entities to DTOs rather than exposing them directly.

### Key Topics

- Paginated list endpoints
- Filtering and sorting via query parameters
- Projecting to DTOs
- Avoiding over-fetching

### Lesson Content

#### 3.1 Paginated List Endpoints

Returning an entire table in one response works fine with 10 rows in development and breaks down completely with 100,000 rows in production. A paginated endpoint accepts page number and page size as query parameters ("/api/products?page=2&pageSize=20"), applies Skip and Take in the LINQ query, and returns both the requested page and the total count so a client can build pagination controls.

```
var products = await _context.Products
    .OrderBy(p => p.Name)
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .ToListAsync();
var totalCount = await _context.Products.CountAsync();
```

#### 3.2 Filtering and Sorting via Query Parameters

Optional query parameters ("?category=electronics&sort=price\_desc") let a client narrow and order results without the API needing a separate endpoint for every possible combination. Each filter should be applied conditionally — only adding a Where clause when that parameter was actually supplied — so the same endpoint serves both an unfiltered browse and a specific filtered search.

#### 3.3 Projecting to DTOs

Returning an EF Core entity directly from an endpoint risks leaking internal fields never meant for a client, and can trigger unwanted lazy-loading of related data. Projecting to a dedicated response DTO with Select — explicitly listing exactly which fields the client should see — keeps the API's public contract intentional and decoupled from the database schema's internal shape.

#### 3.4 Avoiding Over-Fetching

A query that pulls every column and every related entity eagerly, even when the endpoint only needs a product's name and price, wastes database and network resources at scale. Selecting only the fields actually needed — which projecting to a DTO naturally encourages — keeps each endpoint's actual database cost proportional to what it really needs to return.

*An unpaginated "list everything" endpoint is one of the most common causes of a slow, unresponsive API once real data volume arrives. Build pagination in from the very first list endpoint, not as a later optimization.*

### Hands-On Lab: Build the Catalog Endpoint

1. Implement a paginated GET endpoint for your primary catalog resource (e.g. products).
2. Add at least 2 optional filter query parameters (e.g. category, minimum price).



3. Add a sort query parameter supporting at least 2 sort options.
4. Create a response DTO and project your query to it rather than returning entities directly.
5. Test the endpoint in Postman with various combinations of pagination, filtering, and sorting.

#### Tools Used

ASP.NET Core • Entity Framework Core • Postman

## Day 4 — Implementing Core Routes II: Write Operations & Business Logic; Mentor Code Review — 8 hours

### Learning Objectives

- Implement a write operation with real business logic beyond simple field mapping.
- Wrap a multi-step write operation in a database transaction.
- Prepare a clean pull request for mentor code review.

### Key Topics

- Business logic beyond simple CRUD
- Transactions for multi-step operations
- Preparing a clean pull request
- The mentor code review

### Lesson Content

#### 4.1 Business Logic Beyond Simple CRUD

Creating an order isn't just inserting a row — it needs to check product stock availability, calculate line totals and an order total, and decrement stock quantities, all as part of one coherent operation. This is exactly the kind of logic that belongs in a service class rather than directly in the controller, and exactly the kind of logic Week 5's unit tests are best suited to cover thoroughly.

#### 4.2 Transactions for Multi-Step Operations

An order creation that inserts the order, inserts several order items, and decrements stock across multiple products needs all of those steps to succeed or fail together — if stock decrementing fails partway through, the order and its items shouldn't be left behind as inconsistent, order-without-matching-stock-update data. Wrapping the whole operation in an EF Core transaction (“\_context.Database.BeginTransactionAsync()”) guarantees this all-or-nothing behavior.

```
using var transaction = await _context.Database.BeginTransactionAsync();
try
{
    _context.Orders.Add(order);
    foreach (var item in order.Items)
        product.StockQuantity -= item.Quantity;
    await _context.SaveChangesAsync();
    await transaction.CommitAsync();
}
catch { await transaction.RollbackAsync(); throw; }
```

#### 4.3 Preparing a Clean Pull Request

A pull request ready for review contains focused, logically grouped commits with clear messages, a description explaining what the branch adds and any open questions, and passes locally before it's even opened — a reviewer's time is better spent on design and logic feedback than on catching a build error that a local “dotnet build” would have caught first.



#### **4.4 The Mentor Code Review**

The mid-sprint mentor code review looks specifically at API design decisions and business logic correctness — is the transaction boundary drawn correctly, does the stock-check logic handle a concurrent order for the last unit of stock, is the DTO projection appropriately scoped. Structured inline GitHub comments let feedback attach to the exact line it concerns, which is far more actionable than a general comment on the whole pull request.

*A multi-step write operation with no transaction is an operation that can leave the database in a half-finished, inconsistent state the moment any single step fails. If a real transaction boundary should exist and doesn't, that's the single most important thing for a Day 4 code review to catch.*

#### **Hands-On Lab: Implement Order Creation & Open a Pull Request**

1. Implement an order-creation endpoint that checks stock availability and rejects the request if insufficient.
2. Calculate line totals and an order total as part of the creation logic.
3. Wrap the order creation, order items, and stock decrement in a single database transaction.
4. Push your Sprint 1 branch and open a pull request with a clear description of what it adds.
5. Request your mentor as a reviewer and address any feedback received.

#### **Tools Used**

Entity Framework Core • ASP.NET Core • GitHub

### **Day 5 — Sprint Review, Postman Demo & Retrospective — 8 hours**

#### **Learning Objectives**

- Demo a completed sprint's API features clearly via Postman.
- Participate in a structured Sprint Review.
- Write a Sprint Retrospective that produces one concrete action for Sprint 2.

#### **Key Topics**

- Demoing the Sprint 1 API in Postman
- Running the Sprint Review
- Documenting the iteration backlog
- Writing the Sprint Retrospective

#### **Lesson Content**

##### **5.1 Demoing the Sprint 1 API in Postman**

The demo walks the mentor through the sprint's completed features using real Postman requests against the running API — the paginated catalog endpoint with a filter applied live, an order created end-to-end with the resulting stock decrement visible in the database. A live demo against a real running API is far more convincing evidence of “done” than a description of what the code should do.

##### **5.2 Running the Sprint Review**

Per the program's acceptance criteria, a task is only considered done when its endpoint returns correct status codes, the code was reviewed and approved via pull request, it has a passing test covering the happy path and an error case, and there are no unhandled exceptions or known vulnerabilities in the submitted code. Any backlog task not meeting all of these moves to Sprint 2's backlog rather than being counted as finished.



### 5.3 Documenting the Iteration Backlog

Anything raised in Day 4's mentor code review or the Day 5 Sprint Review that wasn't addressed immediately gets logged as a Sprint 2 backlog item, tagged with enough context that it's actionable weeks later without needing to remember the original conversation. Nothing raised in review should quietly disappear.

### 5.4 Writing the Sprint Retrospective

A retrospective stays short and specific: what went well this sprint, what didn't, and one concrete action for Sprint 2 — not a vague “be more careful,” but something like “write the transaction-wrapped write operation's tests before the endpoint itself next time, not after.” The value of a retrospective comes entirely from that action actually being followed in the next sprint's Day 1 planning.

*A retrospective that only lists what went well skips the useful half. The improvement identified here is what makes Sprint 2 measurably better than Sprint 1, not just another week of the same process.*

### Hands-On Lab: Sprint 1 Close-Out

1. Demo your Sprint 1 API to a mentor or peer via Postman: catalog browsing and a full order creation.
2. Check each backlog task against the acceptance criteria and move anything incomplete to a Sprint 2 backlog.
3. Log any unresolved code review feedback as a tagged Sprint 2 backlog item.
4. Write a Sprint 1 Retrospective: what went well, what to improve, and one concrete action for Sprint 2.
5. Assemble a Sprint 1 summary in Notion: ERD, migration history, and a link to your merged pull request — ready for the mentor check-in.

### Tools Used

Postman • Notion or Trello

## 6. Week 6 Deliverables

---

- A Sprint 1 plan: goal statement, sized backlog, and a documented ERD for the full capstone schema.
- An EF Core data model with Fluent API configuration, seed data, and an applied initial migration.
- A paginated, filterable, sortable catalog endpoint projecting to DTOs.
- An order-creation endpoint with real business logic wrapped in a database transaction.
- A reviewed and merged pull request with mentor feedback addressed.
- A written Sprint 1 retrospective with one concrete action for Sprint 2.

## 7. Week 6 Evaluation Criteria

---

Scored by the assigned mentor at the Sprint 1 review using the criteria below (extracted from the program's 100-point internal rubric).



| Criterion                     | 50-69 Developing                                  | 70-84 Proficient                                            | 85-100 Excellent                                           |
|-------------------------------|---------------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------|
| Database schema & SQL quality | Basic tables, no relations or indexing            | Normalized schema, JOINS used correctly, EF Core migrations | Optimized schema, indexes, ERD documented and kept current |
| API design & REST principles  | Basic routes, wrong HTTP verbs/codes              | RESTful, correct status codes, paginated                    | Fully RESTful, consistent conventions, well-projected DTOs |
| Feature completeness          | Less than 50% of sprint features working          | 70-84% of sprint features working end-to-end                | All sprint features delivered and fully functional         |
| C# & .NET usage               | Basic EF Core usage, no transactions where needed | Correct use of transactions and Fluent API                  | Idiomatic, production-grade EF Core patterns throughout    |
| Code quality & structure      | Basic structure, inconsistencies                  | Clean, readable, consistent patterns                        | Production-grade, modular, well-organised                  |
| Attendance & punctuality      | 3-6 absences                                      | 1-2 absences, on time                                       | Perfect attendance, proactive                              |

## 8. Technical Stack — Week 6

| Category          | Tools                                    |
|-------------------|------------------------------------------|
| ORM & Database    | Entity Framework Core, SQL Server        |
| API Framework     | ASP.NET Core                             |
| API Testing       | Postman                                  |
| Version Control   | Git, GitHub (pull requests, code review) |
| Sprint Management | Notion or Trello                         |

## 9. Notes & Best Practices

Sprint 1 establishes the schema and core routes every remaining sprint builds directly on top of — a rushed or poorly normalized schema here creates friction in every following week, exactly the way a poorly structured wireframe set creates friction for the design track's later phases. Week 7 is Sprint 2: layering authentication, role-based access control, and additional middleware onto this week's foundation.